Connecting the Ayur Store to Shopify

The store reads everything live from the Shopify store **avvvsglobal.myshopify.com** through the Storefront API.

What works with no setup. It runs today using Shopify's tokenless access:

- products, collections and search
- the cart, and Shopify checkout

What the steps below switch on:

- medicine information (metafields)
- stock counts
- customer accounts
- prescription uploads
- back-in-stock alerts
- instant cache refresh

Put every value in `store/.env.local` for local work, and in the Vercel project's environment variables for production. `.env.example` lists every variable.

1. Storefront API token (Headless channel)

1. In Shopify admin, open **Sales channels** → **Add** → **Headless** and install it.
2. In **Headless**, click **Create storefront** and name it "Ayur Store".
3. Open **Storefront API** → **Manage permissions**. Turn on:
   - `unauthenticated_read_product_listings`
   - `unauthenticated_read_product_inventory`
   - `unauthenticated_read_metafields`
   - `unauthenticated_read_content`
   - `unauthenticated_write_checkouts` / cart
4. Copy the tokens into the environment:
   - the **private access token** goes in `SHOPIFY_STOREFRONT_PRIVATE_TOKEN`. It is only used on the server.
   - the **public access token** goes in `SHOPIFY_STOREFRONT_PUBLIC_TOKEN`.

2. Customer accounts (Customer Account API)

> **Do this first; checkout is blocked until you do.** The live store currently requires shoppers to log in to a *classic* account (email and password) before checkout. When tested, the store's checkout button landed on `avvvsglobal.com/account/login` instead of checkout. In **Settings → Customer accounts**, set sign-in to **optional** (not required) so guests can check out.

1. Go to **Settings → Customer accounts** and choose **Customer accounts** (the new, passwordless type).
2. In **Headless → your storefront → Customer Account API → Application setup**:
   - **Client type:** Public.
   - **Callback URI:** `https://shop.avvvs.com/account/callback`
   - **JavaScript origin:** `https://shop.avvvs.com`
   - **Logout URI:** `https://shop.avvvs.com`
3. Copy the **Client ID** into `SHOPIFY_CUSTOMER_ACCOUNT_CLIENT_ID`.
4. Set `SESSION_SECRET` to 32 or more random characters (`openssl rand -base64 32`).
5. `SHOPIFY_CUSTOMER_ACCOUNT_DOMAIN` stays `avvvsglobal.com`. It is the shop's primary domain, where the sign-in discovery documents live.

Shopify does not allow `localhost` callbacks. To test sign-in on your own machine, run a tunnel (`ngrok http 3001`) and do two things:

- add the tunnel's HTTPS address as a second callback and origin in Shopify
- set `NEXT_PUBLIC_SITE_URL` to that address

Wishlist on the account (optional)

To make the wishlist follow a signed-in customer across devices, add a definition under **Settings → Custom data → Customers**:

- namespace and key: `custom.wishlist`
- type: **JSON**
- Customer Account API access: **Read and write**

Without it, the wishlist still works in each browser on its own.

3. Medicine information (product metafields)

Under **Settings** → **Custom data** → **Products**, add these definitions. Use namespace `pharmacy` for each, and turn on **Storefront API access** for each.

Key	Type	Shown as
`indications`	Multi-line text	Indications
`dosage`	Multi-line text	Dosage
`anupana`	Single-line text	Anupana (taken with)
`ingredients`	List of single-line text	Ingredients
`contraindications`	Multi-line text	Cautions
`storage`	Single-line text	Storage
`pack_size`	Single-line text	Pack size
`shelf_life`	Single-line text	Shelf life
`licence_no`	Single-line text	Manufacturing licence
`malayalam_name`	Single-line text	Name in Malayalam, under the title (also searchable)
`sanskrit_name`	Single-line text	Name in Sanskrit, under the title
`reference_text`	Single-line text	Classical reference (e.g. Ashtanga Hridayam)
`rx_recommended`	True or false	"Best taken on a doctor's advice" notice

A section appears on the product page only when its metafield has a value. **The store never writes medical information of its own.** Fill these in from AVVVS's approved product literature.

4. Shop by concern (product tags)

The store places medicines under the twelve concerns using **product tags** in Shopify:

- A medicine tagged `concern-pain-management` appears under Pain management.
- A medicine can carry several concern tags.

Until products are tagged. The store uses a starting map worked out from AVVVS's own product descriptions (`src/content/concernMap.ts`). It switches to tags concern by concern: as soon as any product carries, say, `concern-skin-and-hair` , that concern is driven by tags alone.

To move the starting map into Shopify:

1. Have AVVVS's pharmacist review the map. It is also listed in `docs/CONTENT_MISTAKES.md` .
2. Create a custom app with `read_products` and `write_products` scopes (**Settings → Apps → Develop apps**). Put its Admin API token in `.env.local` as `SHOPIFY_ADMIN_TOKEN` .
3. Run `npm run tag-concerns` to see what would change, then `npm run tag-concerns -- --apply` . The script only adds tags; it never removes them.

From then on, edit the tags in Shopify; no code change is needed.

Tag	Concern
`concern-pain-management`	Pain management
`concern-digestive-health`	Digestive health
`concern-respiratory`	Respiratory care
`concern-skin-and-hair`	Skin & hair
`concern-womens-health`	Women's health
`concern-childrens-health`	Children's health
`concern-fever-and-immunity`	Fever & immunity
`concern-diabetes-and-metabolic`	Diabetes & metabolic
`concern-heart-and-circulation`	Heart & circulation
`concern-liver-and-kidney`	Liver & kidney
`concern-mind-sleep-nerves`	Mind, sleep & nerves
`concern-rejuvenation`	Rejuvenation (Rasayana)

4b. Care kits (optional)

A care kit is a set of medicines your pharmacist suggests buying together. Customers can add a whole kit to the cart in one click.

1. In Shopify, go to **Products → Collections → Create collection** and choose *Manual*.
2. Add the medicines and write a short description.
3. Give it a **URL handle** that starts with `kit-`.
   - To show the kit on a concern page, start the handle with that concern's tag name, minus the `concern-`. For example, `kit-pain-management-knee-care` appears on Pain management.
4. The kit then appears on `/kits`, and on the matching concern page.

Until the first kit exists, `/kits` says that kits are coming soon.

4c. Search by symptom

The store's search understands everyday words such as "back pain", "hair fall" and "constipation":

- It suggests the matching concern pages.
- It suggests medicines whose traditional uses mention those words.

The word list lives in `src/content/synonyms.ts`. It can only surface uses already written in `src/content/medicines.ts`, so it adds no new claims. Please include it in the pharmacist's review.

5. Webhooks (instant updates)

Pages refresh from Shopify every 5 minutes on their own. For instant updates, add webhooks so edits show at once:

1. Go to **Settings → Notifications → Webhooks**.
2. Create the following webhooks, each in JSON format and each pointing at `https://shop.avvvs.com/api/revalidate?secret=<SHOPIFY_REVALIDATE_SECRET>`:
   - Product creation, Product update, Product deletion
   - Collection creation, Collection update, Collection deletion
3. Set `SHOPIFY_REVALIDATE_SECRET` to a long random string.

6. Prescriptions, stock alerts, mail

- **Prescription uploads.** Create a Vercel Blob store and set `BLOB_READ_WRITE_TOKEN`.
  - Each upload is saved under a random name, and its link is added to the order's *Additional details* (note attributes) as `Prescription`.
  - These links are unguessable, but anyone holding one can open the file.
  - Until the token is set, the upload box tells customers to email the prescription instead.

- **Back-in-stock alerts.** Set `DATABASE_URL` (Neon Postgres, the same kind as `web/`). Requests are stored in a `stock_alerts` table.
- **Mail to the pharmacy.** Use the same SMTP variables as `web/` (`SMTP_*`, `MAIL_FROM`, `MAIL_TO`).

7. Payments, shipping and taxes

These live entirely in Shopify's checkout. The store sends the cart there and never handles payment details. Set up the following in Shopify admin:

- payment providers (e.g. Razorpay or PayU for UPI, and COD)
- shipping zones and rates
- GST

8. Domain and deploy

1. Create a new Vercel project from this repository (`avvvspharmacy`). It uses the Next.js preset.
2. Add the environment variables from the steps above.
3. Point `shop.avvvs.com` at Vercel (a CNAME to `cname.vercel-dns.com`).

The main site's "Ayur Store" links already go to `https://shop.avvvs.com/`.

Optional extras:

- set `NEXT_PUBLIC_FREE_SHIPPING_THRESHOLD` (in INR) to show a free-delivery progress bar in the cart. Match it to your Shopify shipping rule.
- set `NEXT_PUBLIC_WHATSAPP` to change the WhatsApp number (digits with country code).